

CPSY 1291 — Bootcamp Notes

The Python and mathematics you will actually use

Fall 2026 · read on your own; the recitations work through the same material
scoped to what Assignment 1 actually requires

How to use these notes

This is not a tutorial and not a reference manual. It is a *map*: it tells you which ideas you need, what each one is for, and roughly what it looks like in code. Read it once before the term starts, then come back to whichever section an assignment has just made relevant.

Everything here was chosen by working backwards from **Assignment 1**: if a tool appears in these notes, you will use it in the first few weeks. Ideas the course needs later — gradients, the chain rule, how networks are trained — are deliberately absent, and arrive with the assignment that first needs them.

If you already program comfortably in Python and have had linear algebra, skim the boxes and skip the rest. If you are arriving from Java, Racket, or MATLAB, Sections 1–2 are the ones to read slowly.

The one rule. Before you write a line of code, you should be able to say in one sentence what it is supposed to compute. If you cannot, the thing you are stuck on is the *idea*, not the syntax — and no amount of fiddling with brackets will fix it. Come back to the idea first.

A note on AI tools, since it would be strange not to address it. In this course, assignments are your own work and you should not use an LLM to write them. Outside this course — in research, in industry — you will use these tools, and so do we. Both of those are true at once because what is worth learning here is not the typing. It is knowing what to compute, what shape the answer should be, and how to tell when it is wrong. That judgment is what makes generated code useful rather than dangerous, and it is what these notes and the assignments are built to develop. Accordingly, the emphasis throughout is on *what a computation means and how to check it*, and rather less on syntax.

Sections: (1) Python essentials (2) NumPy and the shape discipline (3) Vectors and matrices (4) Dissimilarity matrices (5) The library calls you will make (6) Correlation and p -values (7) Plots as instruments (8) A debugging recipe (9) Exercises (10) Cheat sheet

1 Python essentials

1.1 Values, containers, and the two you will confuse

A list is a general-purpose sequence that can hold anything. A NumPy array is a block of numbers of one type, with a shape. Almost all numerical work in this course uses arrays; lists are for bookkeeping.

```
xs = [3, 1, 4] # list: fine for a handful of things
xs.append(1) # lists grow
d = {'cat': 0, 'dog': 1} # dict: look things up by name
d['cat'] # -> 0
for name, idx in d.items():
    print(name, idx)
```

A tuple (a, b) is an immutable list; you will meet them mostly as array shapes and as multiple return values.

1.2 Functions, and why yours should return rather than print

```
def euclidean(a, b):
    """Straight-line distance between two points."""
    return ((a - b) ** 2).sum() ** 0.5
```

A function that prints its answer cannot be reused; one that returns it can. Write a doc-string saying what goes in and what comes out — when you come back in a week, it is the only thing that will still be true.

1.3 Loops, and the habit to unlearn

Python loops are correct and slow. In numerical code they are also usually a sign that you are thinking one item at a time when the operation applies to everything at once. Section 2 is about the alternative.

```
sq = [x**2 for x in xs] # comprehension: a loop that makes a list
for i, x in enumerate(xs): # when you need the index too
    print(i, x)
for a, b in zip(names, values): # two sequences in step
    print(a, b)
```

1.4 Reading an error message

Read the *last* line first: it names the error. Then read upward to find the line of *your* code that caused it. Three you will meet constantly:

- `ValueError: operands could not be broadcast together with shapes (120,4096) (4096,120)` — an array is the wrong way round. See §2.4.
- `IndexError: index 120 is out of bounds for axis 0 with size 120` — Python counts from 0, so the last valid index is 119.
- `TypeError: 'numpy.float64' object is not callable` — you wrote `f(x)` where `f` is a number, usually because a name got reused.

2 NumPy and the shape discipline

This is the section that pays for itself. Nearly everything in this course is “do the same arithmetic to a few thousand vectors”, and NumPy is how that is said.

2.1 Shape is your unit test

Every array has a shape: a tuple of its dimensions. **Print shapes constantly.** Most bugs in numerical code are shape bugs, and they announce themselves immediately if you look.

```
X.shape # e.g. (120, 4096): 120 stimuli, 4096 features each
X.dtype # float64, uint8, ...
X.ndim # 2
```

The course convention. Data matrices are **stimuli by features**: one row per image, one column per pixel, unit, or voxel. So `X[i]` is the vector for stimulus *i*, and `X.shape[0]` is how many stimuli you have. Every assignment assumes this.

2.2 Indexing and slicing

```
X[0] # first row: the feature vector of stimulus 0
X[:, 3] # column 3: feature 3 across all stimuli
X[:5] # first five rows
X[::24] # every 24th row -- useful when the data repeats
X[-1] # last row
```

Slicing never copies; it gives you a view onto the same memory. Modifying a slice modifies the original, which is efficient and occasionally surprising.

2.3 The axis argument

The axis you name is the axis that disappears. `X.sum(axis=0)` sums down the rows and leaves one number per column. `X.mean(axis=1)` averages across the columns and leaves one number per row.

```
X.shape # (120, 4096)
X.mean(axis=0).shape # (4096,) -- the average stimulus
X.mean(axis=1).shape # (120,) -- each stimulus's average feature value
X.mean(axis=1, keepdims=True).shape # (120, 1) -- kept for broadcasting
```

2.4 Broadcasting

When two arrays have different shapes, NumPy stretches size-1 dimensions to match. This is what lets you subtract one vector from a whole matrix:

```
X - X.mean(axis=0) # (120,4096) - (4096,) -> centers each column
X - X.mean(axis=1, keepdims=True) # (120,4096) - (120,1) -> centers each row
```

Note that those two do genuinely different things, and `keepdims` is what selects between them. The rule: compare shapes from the right; dimensions must be equal, or one of them must be 1.

The idiom worth memorizing, because it turns any pairwise computation into one line:

```
diff = X[:, None, :] - X[None, :, :] # (120, 1, 4096) - (1, 120, 4096)
# -> (120, 120, 4096)
D = (diff ** 2).sum(-1) ** 0.5 # (120, 120) all pairwise distances
```

That allocates a large intermediate array. When it is too large, use the algebraic route instead — see §3.3.

2.5 Masks, counting, and sorting

```
mask = (labels == 2) # boolean array, one entry per stimulus
X[mask] # just the rows where mask is True
mask.sum() # how many -- True counts as 1
np.unique(labels, return_counts=True) # what values, how many of each
np.argsort(v)[:5] # indices of the five smallest values
np.argsort(v)[::-1][:5] # indices of the five largest
np.triu_indices(N, k=1) # the upper triangle: each pair exactly once
```

`argsort` is how you get “the five most similar images”, and `triu_indices` is how you compare two matrices of pairwise values without counting each pair twice.

3 Vectors and matrices

3.1 A list of numbers is a point

The central move of this whole course: if you write down D numbers about something — the pixel values of an image, the firing rates of D neurons, a person’s rating on D scales — you can treat that list as a *point* in a D -dimensional space. The lectures write the vector for stimulus i as $\mathbf{x}^{(i)}$ and its j -th entry as $x_j^{(i)}$; N stimuli make an $N \times D$ matrix $\mathbf{X}$, one row per stimulus. Two things are “similar” when their points are close. Everything else is detail about what *close* should mean.

3.2 Length, distance, and the dot product

For two stimuli $\mathbf{a} = \mathbf{x}^{(i)}$ and $\mathbf{b} = \mathbf{x}^{(k)}$ in $\mathbb{R}^D$ (the lectures keep the superscripts; here $\mathbf{a}, \mathbf{b}$ are shorter):

$$\|\mathbf{a}\| = \sqrt{\sum_j a_j^2}, \quad d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|, \quad \mathbf{a} \cdot \mathbf{b} = \sum_j a_j b_j = \|\mathbf{a}\| \|\mathbf{b}\| \cos \phi.$$

The dot product is the workhorse. Read it as: *how much of a points along b* , scaled by both lengths. It is large when two vectors agree in direction, zero when they are perpendicular, negative when they oppose.

```
a @ b # dot product
np.linalg.norm(a) # length
np.linalg.norm(a - b) # distance
```

3.3 Cosine, and the identity you will use

Dividing out the lengths leaves only the angle ϕ between the two vectors:

$$\cos \phi = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

This ignores magnitude, which is what you want when overall brightness or overall firing rate is not the thing you care about.

Worth knowing: Pearson correlation is exactly cosine similarity after subtracting each vector's own mean. So “correlation distance” is “cosine distance on centered vectors”, and any code you have for one gives you the other for free.

The lectures name the three dissimilarities this way, and Assignment 1 §1b asks you to write all three as `euclidean(X)`, `cosine_dist(X)` and `correlation_dist(X)`, each returning the $N \times N$ matrix over the rows of X :

$$d_{\text{euc}} = \|\mathbf{a} - \mathbf{b}\|, \quad d_{\text{cos}} = 1 - \cos \phi, \quad d_{\text{corr}} = 1 - r,$$

where r is the Pearson correlation of $\mathbf{a}$ and $\mathbf{b}$: the cosine after each vector's own mean is subtracted. The last two lie in $[0, 2]$.

Two ways to get all pairwise distances. The second is the one to use at scale, and it follows from expanding $\|\mathbf{a} - \mathbf{b}\|^2 = \mathbf{a} \cdot \mathbf{a} + \mathbf{b} \cdot \mathbf{b} - 2 \mathbf{a} \cdot \mathbf{b}$:

```
sq = (X**2).sum(axis=1) # (N,)
D2 = sq[:, None] + sq[None, :] - 2 * (X @ X.T)
D = np.sqrt(np.maximum(D2, 0)) # clip: tiny negatives are roundoff
```

3.4 Matrices, two ways to read them

A matrix is a grid of numbers, and which meaning you want depends on context:

- **As data.** X with one row per stimulus and one column per feature. This is the reading used above.
- **As a map.** W turns a vector into another vector, $\mathbf{z} = W\mathbf{a}$. Row j of W is a vector whose dot product with $\mathbf{a}$ gives z_j . This is what a layer of a neural network does, and it is how we will write them all semester.

Matrix multiplication AB requires the inner dimensions to match: $(m \times k)(k \times n) \rightarrow (m \times n)$. Check this by reading the shapes aloud before you write the line. The transpose $X.T$ swaps rows and columns and is usually how you make the inner dimensions agree.

3.5 Centering, variance, covariance

Subtracting the mean of each feature puts the cloud of points at the origin without changing its shape. After centering, the covariance matrix

$$C = \frac{1}{N-1} \tilde{X}^T \tilde{X}$$

holds the variance of each feature on its diagonal and, off the diagonal, how each pair of features vary together.

3.6 Eigenvectors, and what PCA is doing

For a square matrix C , an *eigenvector* v is a direction that C does not rotate — it only stretches it:

$$Cv = \lambda v,$$

where the *eigenvalue* λ says by how much. Applied to a covariance matrix, this has a very concrete reading:

PCA in one sentence. Center the data, form its covariance matrix, and take that matrix's eigenvectors: they are the directions along which the data varies most, and each eigenvalue is how much variance lies along its direction.

So “the first principal component” is just the single direction in feature space along which your points are most spread out, and “variance explained” is the fraction of total spread you keep if you describe each point only by where it falls along the first few such directions. In practice you will call PCA from scikit-learn rather than doing the eigendecomposition yourself — but if you cannot say what the components *are*, the output is just numbers.

4 Dissimilarity matrices

Almost every analysis in the first assignment consumes the same object: a square matrix D holding the dissimilarity of every pair of stimuli. It is worth being precise about it, because three separate conventions collide here.

4.1 Square form and condensed form

A dissimilarity matrix over N stimuli is $N \times N$, symmetric, with zeros on the diagonal; entry d_{ik} is the dissimilarity of stimuli i and k (the lectures call it the RDM, representational dissimilarity matrix). That is N^2 numbers to store $N(N-1)/2$ facts, so SciPy also has a *condensed* form: a flat vector of just the upper triangle. `pdist` returns condensed; `squareform` converts between the two, in both directions.

```
d = pdist(X, metric='euclidean') # condensed: (N*(N-1)/2,)
D = squareform(d) # square: (N, N)
squareform(D, checks=False) # back to condensed
```

Functions disagree about which they want. `linkage` expects condensed; `imshow` needs square; MDS with precomputed dissimilarities wants square. When a call complains about your distance matrix, this is almost always why.

4.2 Comparing two matrices

To ask whether two systems organize the same stimuli the same way, you compare their dissimilarity matrices entry by entry. Take each one's upper triangle so that every pair is counted exactly once, and correlate the two lists:

```
iu = np.triu_indices(N, k=1) # k=1 excludes the diagonal
spearmanr(D1[iu], D2[iu])
```

Using the whole matrix instead would count every pair twice and add N meaningless zeros, which drags the correlation upward for no reason.

4.3 Checks worth running every time

```
D.shape # (N, N)?
np.allclose(D, D.T) # symmetric? (1e-16 off is roundoff, not a bug)
np.allclose(np.diag(D), 0) # zeros on the diagonal?
(D >= 0).all() # nonnegative?
np.isnan(D).sum() # any missing?
np.fill_diagonal(D, 0) # force the diagonal if it drifted
```

5 The library calls you will actually make

You are not expected to implement MDS or t-SNE. You are expected to know what they take, what they return, and what could go wrong — so this section is about the *pattern*, not the arguments.

5.1 The scikit-learn pattern

Nearly every scikit-learn tool works the same way: construct an object with its settings, then `fit_transform` your data to get the result. Anything the method learned is then stored on the object, with a trailing underscore.

```
pca = PCA(n_components=50)
Y = pca.fit_transform(X) # (n_stimuli, 50)
pca.explained_variance_ratio_ # (50,) fraction of variance per component
np.cumsum(pca.explained_variance_ratio_) # running total
```

The same shape of call gives you `MDS(...)` and `TSNE(...)`. Two things to watch. Some of these take a *dissimilarity matrix* rather than feature vectors, which you signal with `metric='precomputed'`. And some are stochastic, so they take a `random_state` — meaning two runs give two answers unless you fix it. The call the lectures use:

```
mds = MDS(n_components=2, metric='precomputed', n_init=8,
          normalized_stress='auto', random_state=0)
Y = mds.fit_transform(D) # (N, 2); mds.stress_ afterwards
```

Add `metric_mds=False` for non-metric MDS, which fits only the rank order of the dissimilarities. `metric='precomputed'` is the scikit-learn ≥ 1.8 spelling (`dissimilarity='precomputed'` before); the Assignment 1 setup cell installs 1.8 for you.

5.2 Hierarchical clustering

```
Z = linkage(squareform(D, checks=False), method='average')
dendrogram(Z, no_labels=True) # draw the tree
grp = fcluster(Z, 6, criterion='maxclust') # cut it into 6 groups
```

`linkage` builds the whole tree; `fcluster` cuts it at a chosen height or into a chosen number of groups. The tree encodes structure at every level at once, so **choosing where to cut is choosing which level you want to talk about** — not discovering how many clusters there “really” are.

5.3 Fitting a curve, and what R^2 means

```
c = np.polyfit(x, y, 1) # straight line: returns coefficients
yh = np.polyval(c, x) # the fitted values
p, _ = curve_fit(lambda x, a, b: a*np.exp(-b*x), x, y) # a nonlinear fit
r2 = 1 - ((y - yh)**2).sum() / ((y - y.mean())**2).sum()
```

R^2 is the fraction of the variance in y that your fit accounts for. Read that definition carefully: it is a *ratio*, so it depends on how variable y was to begin with. Averaging your data into bins before fitting removes noise from the bottom of that fraction and pushes R^2 up without the fitted curve changing at all. **Two R^2 values are comparable only if they were computed against the same target.**

6 Correlation, ranks, and honest p -values

- **Mean, variance, standard deviation.** The center, the spread, and the spread in the original units. A feature with zero variance is a special hazard: correlation divides by a standard deviation, so if you z -score each *unit* across stimuli (Assignment 1 §3h) a unit that never responds divides by zero and puts a silent nan into everything downstream. Correlation *between two images* across their units (§1b) does not fail on a dead unit, but it is biased by it: see exercise 10.

- **The z -score** $(x - \mu)/\sigma$ puts different measurements on a common scale. Note that z -scoring each vector and then taking dot products is computing correlations.
- **Pearson vs. Spearman correlation.** Pearson (r) measures how well a *straight line* fits; Spearman (ρ) does the same after replacing values by their ranks, so it measures whether two things increase together without assuming the relation is linear. Use Spearman when only the ordering is meaningful.
- **What a p -value assumes.** A p -value asks how surprising your result would be if nothing were going on — but it computes that using your sample size n . If your rows are not independent (say, 24 photographs of the *same* object), then your effective n is far smaller than the number of rows, and the p -value is correspondingly overconfident. **Ask what the independent unit is before reporting a p -value.**
- **Null baselines beat thresholds.** Rather than comparing a number against a rule of thumb, shuffle your data, redo the analysis, and see what values arise by chance. A result that beats its own shuffled baseline is meaningful in a way that “above 0.7” never is.

7 Plots as instruments

Plots in this course are not decoration; they are how you check that an analysis is doing what you think.

```
fig, ax = plt.subplots(2, 3, figsize=(12, 7)) # a grid of panels
ax[0, 1].imshow(D, vmin=..., vmax=...) # a matrix as an image
ax[1, 0].scatter(Y[:, 0], Y[:, 1], s=8, c=labels)
ax[0, 0].set_title('...'); plt.tight_layout(); plt.show()
```

Two habits. First, **look at your raw data before computing anything from it** — display some images, print the range, count the zeros. Second, when a matrix plot looks washed out, a few extreme values are eating the color scale; set `vmin` and `vmax` from percentiles instead of letting matplotlib choose.

8 A debugging recipe

When something is wrong, work through this in order. It resolves most problems before you have to think hard.

1. **Shape.** Print it. Is it what you expect, and is it the right way round?
2. **Type and range.** `X.dtype`, `X.min()`, `X.max()`. Integer division and `uint8` overflow cause quiet, plausible-looking wrongness.
3. **Missing values.** `np.isnan(X).sum()`. One nan spreads through everything it touches.
4. **Degenerate rows or columns.** Anything with zero variance breaks a correlation taken along that axis, because you divide by its standard deviation.

5. **A case you can check by hand.** Compute one value with a calculator, or on a 2×2 example, and compare.
6. **Plot it.** A picture of a wrong matrix usually looks obviously wrong.

9 Exercises

Short, and meant to be done at a keyboard. Answers on the last page.

1. `X` has shape `(120, 4096)`. What are the shapes of `X.mean(0)`, `X.mean(1)`, and `X.mean(1, keepdims=True)`? Which would you use to center each stimulus's own vector?
2. Write one line giving the index of the stimulus most similar to stimulus 7, given a similarity matrix `S`. (Careful: what is `S[7, 7]`?)
3. `A` is `(120, 4096)` and `B` is `(4096, 10)`. Which of `A @ B`, `B @ A`, `A.T @ B` is legal, and what shape does it give?
4. Two vectors are $\mathbf{a} = (1, 0)$ and $\mathbf{b} = (3, 0)$. What is their cosine similarity? Their Euclidean distance? What does the disagreement tell you?
5. You have a 120×120 dissimilarity matrix and want to correlate it against another one. How many numbers should go into that correlation, and which NumPy call gets them?
6. `linkage` raises an error when you hand it your 120×120 matrix. What is wrong, and what is the fix?
7. A published paper reports $R^2 = 0.96$ for the same relationship where you got 0.84. Before concluding your analysis is worse, what should you check?
8. You have 1,224 images that are 51 objects photographed from 24 angles. You correlate a property across all 1,224 and get $p < 10^{-40}$. Why is that misleading, and what would you report instead?
9. A matrix that should be symmetric has `np.abs(D - D.T).max() == 3e-17`. Is something wrong?
10. Write, without a loop, the fraction of entries in `X` that are exactly zero. Why might you care?

10 Cheat sheet

Call	What it is for
getting data in, and looking at it	
<code>np.load(path)</code>	read .npz; index it like a dict
<code>X.shape, X.dtype</code>	the first thing to print, always
<code>np.isnan(X).sum()</code>	count missing values
<code>np.unique(v, return_counts=True)</code>	what values occur, how often
<code>X.min(), X.max(), np.ptp(X)</code>	range, and range as one number
reshaping and selecting	
<code>X[::24]</code>	every 24th row
<code>X[mask]</code>	the rows where a boolean is True
<code>X.reshape(N, -1)</code>	flatten all but the first axis
<code>np.argsort(v)</code>	indices that would sort v
<code>np.triu_indices(N, k=1)</code>	each pair once, no diagonal
the linear algebra	
<code>X @ X.T</code>	every pairwise dot product at once
<code>X[:, None, :] - X[None, :, :]</code>	every pairwise difference at once
<code>np.linalg.norm(X, axis=1)</code>	the length of each row
<code>X - X.mean(0)</code>	center each feature (column)
<code>X - X.mean(1, keepdims=True)</code>	center each stimulus (row)
dissimilarity matrices	
<code>pdist(X, metric=...)</code>	condensed pairwise distances
<code>squareform(d)</code>	condensed ↔ square
<code>np.fill_diagonal(D, 0)</code>	force the diagonal to zero
the methods	
<code>PCA(...).fit_transform(X)</code>	principal components
<code>.explained_variance_ratio_</code>	variance carried by each one
<code>MDS(...), TSNE(...)</code>	same pattern; watch <code>random_state</code>
<code>linkage(...), fcluster(...)</code>	build a tree, then cut it
<code>spearmanr(a, b)</code>	rank correlation
<code>np.polyfit / curve_fit</code>	fit a line / a nonlinear curve
plotting	
<code>plt.subplots(r, c)</code>	a grid of panels
<code>plt.imshow(D, vmin=, vmax=)</code>	look at a matrix; set limits by percentile
<code>plt.scatter(Y[:,0], Y[:,1])</code>	look at a 2-D embedding
<code>OffsetImage / AnnotationBbox</code>	draw images at their coordinates

Answers

1. `(4096,)`, `(120,)`, `(120,1)`. Use the third: it broadcasts against `(120,4096)` to subtract each row's own mean.
2. `np.argsort(S[7])[-2]` — the last entry is stimulus 7 itself, since nothing is more similar to it than itself.
3. Only `A @ B` is legal, giving `(120, 10)`. The inner dimensions must match.
4. Cosine similarity 1 (identical direction); Euclidean distance 2. They disagree because cosine ignores magnitude — which is exactly why you would ever choose one over the other.

5. $120 \times 119 / 2 = 7,140$, via `np.triu_indices(120, k=1)`. Using the full matrix counts every pair twice and adds 120 meaningless zeros from the diagonal.
6. `linkage` wants the condensed form, not the square one. Pass `squareform(D, checks=False)`.
7. Whether the two R^2 values were computed against the same target. If theirs was fitted to binned averages and yours to individual pairs, the difference is mostly the binning, not the model.
8. The 24 views of one object are not independent measurements, so the effective sample size is nearer 51 than 1,224. Report the correlation computed on the 51 object means.
9. No. That is floating-point roundoff, around 10^{-17} . Symmetrize with `(D + D.T) / 2` and move on.
10. `(X == 0).mean()`. Network activations are full of exact zeros because of the ReLU nonlinearity. A unit that is *always* zero has no variance, so z -scoring it across stimuli divides by zero; and zeros that two images *share* pull both means down after centering and then count as agreement, which inflates their correlation (Assignment 1 §2c). Decide the unit set once, for every image, and say what you did.